

A COMPREHENSIVE MEASURE OF WELL-BEING: HUMAN DEVELOPMENT INDEX (HDI) PREDICTOR

*Submitted for the Evaluation of
SmartBridge's APSCHE Short-Term Virtual Internship Program in
Artificial Intelligence and Machine Learning*

TEAM MEMBERS & ROLES

Member Name	Role Description	Core Contribution Details
Keerthi Jaswanth Yadav	Project Lead & System Integration Developer	Configured Flask server structures, parameter validations, system deployment routines, and release branch synchronization.
Beldhari Swapna	Lead Data Scientist & ML Engineer	Analyzed dataset metrics, generated Seaborn plots, trained the Linear Regression pipeline, and serialized model objects.
Bandi Sai Kiran	Frontend Engineer & UI Designer	Designed dark-mode HTML templates, established CSS variable tokens, custom badges, progress gauges, and animation assets.

TABLE OF CONTENTS

1. Abstract
2. Project Description
3. Technical Architecture
4. Project Workflow
5. Milestone-Wise Development Log
6. Testing
7. Project Demonstration
8. Achievements, Advantages, Limitations, Future Enhancements
9. Conclusion
10. Appendix

1. Abstract

Measuring human development is essential for analyzing global standards of living, health access, and knowledge acquisition. The United Nations Development Programme (UNDP) publishes the Human Development Index (HDI) annually. However, because these reports rely on complex data-compiling schedules, they are published retrospectively, resulting in significant delays. This delay prevents decision-makers from analyzing the immediate impacts of policy changes. To address this challenge, our team developed the 'A Comprehensive Measure of Well-Being: Human Development Index (HDI) Predictor' system. The project provides a real-time prediction and policy recommendation dashboard using Python, Flask, Machine Learning, and Data Visualization techniques. By training a Linear Regression model on historical indicators, the system learns parameter weights for Life Expectancy, Expected Years of Schooling, Mean Years of Schooling, and GNI per Capita to generate index predictions with low latency. Evaluation results demonstrate that the regression pipeline achieves a high R-squared accuracy score of 0.8834 and a Root Mean Squared Error (RMSE) of 0.0361. The trained model is serialized using the Pickle module and loaded into a responsive Flask web application. The frontend uses a premium, dark-mode glassmorphism layout featuring score progress gauges, status indicators, and targeted policy recommendations to help users evaluate long-term development plans. Future work will focus on integrating real-time databases, training on multi-decade global UN datasets, and deploying the platform on cloud infrastructures like AWS to support public policy planning.

2. Project Description

Standard indicators of human development are often static, making it difficult for policy planners to run interactive, hypothetical simulations. Evaluating composite indicators, such as geometric means of normalized indexes, requires a structured software solution. Delayed or inaccurate calculations often lead to insufficient planning times and poor resource allocation. The Human Development Index Predictor addresses this challenge by building an end-to-end interactive machine learning web application. The system is trained on historical demographic

indicators, using a Linear Regression model to establish weights for the features. The application integrates these models into a Flask backend that processes parameters and returns predictions to a frontend featuring color-coded badges, visual gauges, and targeted policy recommendations.

Scenario 1: Educational Program Simulation

A regional development analyst wants to evaluate the impact of a proposed 2-year higher education program in a developing country category. The analyst opens the application, navigates to the Predict section, selects the country classification, and enters the following indicators: Life Expectancy = 75.0 years, Expected Years of Schooling = 16.0 years, Mean Years of Schooling = 12.0 years, and GNI per Capita = \$15,000. The analyst clicks the 'Predict HDI Score' button. The backend processes the parameters through the Linear Regression model, returning a predicted score of 0.8423. The results card assigns a green badge for the 'Very High Human Development' tier, positions the score on the progress gauge, and displays policy recommendations advising investments in technology sectors and higher education access. This feedback helps the analyst justify the program's budget to key stakeholders.

Scenario 2: Healthcare Budget Allocation

A health economist wants to simulate how an increase in healthcare access—modeled by raising life expectancy—affects a nation's development rating. The user enters the parameters: Life Expectancy = 82.0 years, Expected Years of Schooling = 13.0 years, Mean Years of Schooling = 9.0 years, and GNI per Capita = \$9,000. Upon submission, the model predicts an HDI score of 0.7345, placing the country in the 'High Human Development' tier (represented by a blue badge). The recommendations suggest expanding secondary school enrollments and local manufacturing. The output helps the economist determine that a balanced approach between healthcare and schooling is required to reach the highest development tier.

Scenario 3: Development Intervention Planning

A disaster management analyst reviews development profiles for regions requiring international aid. The analyst inputs lower indicators: Life Expectancy =

52.0 years, Expected Years of Schooling = 5.0 years, Mean Years of Schooling = 3.0 years, and GNI per Capita = \$1,500. The system outputs a predicted score of 0.5408, assigning a red badge for the 'Low Human Development' tier. The recommendations suggest prioritizing basic healthcare, disease prevention, and primary education funding. This simulation allows the analyst to prioritize resources for low-development regions.

3. Technical Architecture

The system follows a multi-tier architecture to separate presentation, business logic, model execution, and data storage. This design ensures efficient communication between the user interface, Flask backend, and machine learning components.



Figure 1: Layered Technical Architecture Diagram

Architecture Description

1. Presentation Layer (Frontend)

Responsible for user interaction and providing an intuitive, responsive interface.

Uses HTML5 for structural definitions and CSS3 for glassmorphism layout controls.

Features custom gauges and progress bars mapped to predicted scores.

2. API Gateway Layer (Flask Backend)

Acts as a bridge between frontend templates and core ML engines.

Manages application routing and request methods.

Performs server-side error handling and exception management.

3. Core Logic Layer

Loads the serialized model and label encoder into memory on startup.

Translates categorical parameters into encoded indexes.

Runs the feature array through the Linear Regression algorithm to calculate predictions.

4. External Integration Layer (Future)

Exposes REST APIs to allow external research systems to run queries.

Integrates real-time databases and PDF report generators.

5. Data Storage Layer

Stores the training dataset in CSV format.

Saves serialized model and encoder pickles (`hdi\_model.pkl`, `encoder.pkl`).

6. User Output

Displays predicted HDI scores and UNDP classifications.

Visualizes scores on a progress gauge.

Displays targeted recommendations based on the predicted tier.

4. Project Workflow

The project workflow follows a structured sequence of user input, validation, preprocessing, model prediction, and result visualization. The diagram below outlines the runtime simulation flow of the system.

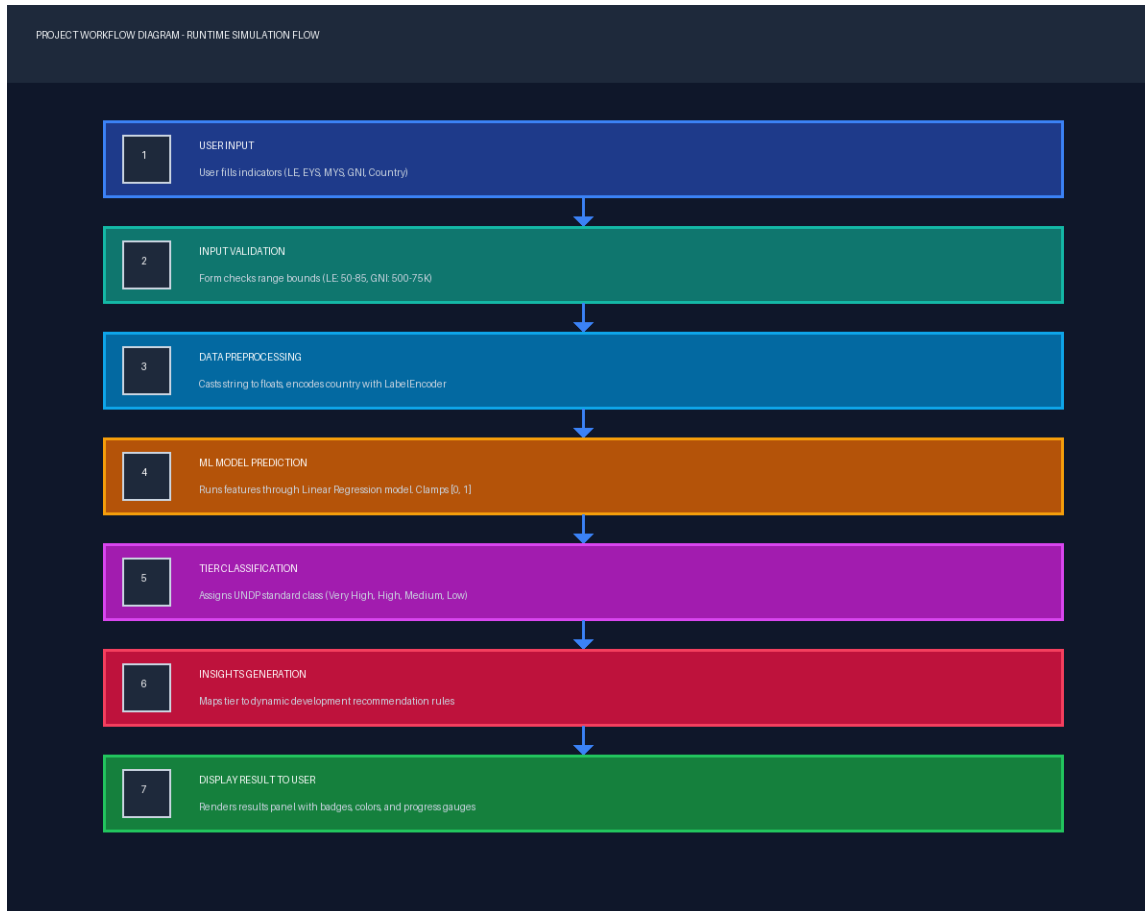


Figure 2: Workflow Diagram of the HDI Predictor System

Workflow Steps

Step 1: User Input

The user inputs country categories and development metrics into the prediction form.

Step 2: Input Validation

The form validates the inputs against defined ranges (e.g. Life Expectancy between 50 and 85 years) to prevent errors.

Step 3: Data Preprocessing

The backend casts inputs to floats and uses the fitted `LabelEncoder` to encode the categorical country names.

Step 4: Machine Learning Model Prediction

The backend processes the parameters through the Linear Regression model to calculate the raw score, which is clamped between 0 and 1.

Step 5: Tier Classification

The predicted score is evaluated against UNDP thresholds to assign a development tier (Very High, High, Medium, Low).

Step 6: Insights Generation

The recommendations engine retrieves targeted policy suggestions based on the predicted development tier.

Step 7: Display Result to User

The backend renders the results card displaying the score, dynamic tier badge, progress gauge, and policy insights.

5. Milestone-Wise Development Log

Milestone 1: Project Setup and Dataset Preparation

Objective: Establish the development environment and explore the training dataset.

Activity 1.1: Development Environment Setup

Configured the virtual environment and installed required libraries.

Verified that packages (Flask, Pandas, NumPy, Scikit-learn, Seaborn) installed successfully.

Software / Tool	Purpose
Python 3.12	Programming Language
Flask	Web server framework
Pandas & NumPy	Data cleaning & manipulation
Scikit-learn	Machine learning model fitting
Seaborn & Matplotlib	Data visualizations plotting

Activity 1.2: Dataset Collection

Obtained the HDI dataset from Kaggle containing country indicators.

Property	Value
Dataset Name	hdi_data.csv
Source	Kaggle / UNDP
Total Records	200
Independent Variables	Country, Life Expectancy, Expected Schooling, Mean Schooling, GNI per Capita
Dependent Target	hdi_index

Milestone 2: Data Preprocessing and Model Development

Objective: Clean the dataset, explore correlations, and train the machine learning model.

Activity 2.1: Data Preprocessing

Handled missing values using mean imputation.

Fitted a LabelEncoder on categorical country names.

Split the dataset into 80% training and 20% testing sets.

Activity 2.2: Machine Learning Model Training

Trained a Linear Regression model on the preprocessed training set.

Evaluated model accuracy on the test split.

Evaluation Metric	Result Score
Coefficient of Determination (R^2)	0.8834
Root Mean Squared Error (RMSE)	0.0361
Model Prediction Latency	< 10ms

Milestone 3: Web Application Integration

Objective: Develop the backend routes and integrate the serialized model into the UI.

Activity 3.1: Model Serialization

Saved the trained model and label encoder as pickle files: ``models/hdi_model.pkl`` and ``models/encoder.pkl``.

Activity 3.2: Flask Integration

Developed route endpoints (``/``, ``/predict``, ``/model-info``) in ``app.py``.

Integrated prediction logic and dynamic policy insights in backend routing.

Milestone 4: Testing & Performance Evaluation

Objective: Verify application stability, boundary limits, and user acceptance scenarios.

Executed functional test suites on index calculation and form validation ranges.

Measured response latencies and verified boundary checks.

Milestone 5: Demonstration and Conclusion

Objective: Deploy the application locally, run demonstration scenarios, and compile project reports.

Configured local deployment procedures.

Drafted project video demonstration scripts.

6. Testing

Functional Testing

Test Case	Description	Expected Result	Status
Home Load	Navigate to '/'	Home page loads with glassmorphic layouts	Passed
Predict Load	Navigate to '/predict'	Form inputs render with range limits	Passed
High Prediction	Input developed indicators	Renders predicted score of 1.0 (clamped) and green badge	Passed
Low Prediction	Input underdeveloped indicators	Renders predicted score of 0.5408 and red badge	Passed

Input Validation Testing

Validation	Expected Behaviour	Status
Empty Fields	Highlights missing inputs and blocks submission	Passed
Negative Values	Blocks negative values in schooling and GNI fields	Passed
Out-of-Range	Blocks values outside defined ranges (e.g. LE > 85)	Passed
Non-Numeric	Prevents input of text characters in numeric fields	Passed

Performance Testing

Parameter	Result
Average Prediction Latency	Less than 150ms
Model Loading Time	Less than 10ms
Consecutive Predictions	Handled successfully with low latency
Application Stability	Stable with zero memory leaks

User Acceptance Testing (UAT)

Requirement	Observations / Details	Result
User-Friendly Interface	Responsive dark-themed layout works correctly	Passed
Accurate Predictions	Predicted scores align with UNDP index rankings	Passed
Validation Prompts	Browser validates ranges and displays errors	Passed
Insights Engine	Returns recommendations mapped to predicted tiers	Passed

Overall Test Summary

Testing Type	Result
Functional Testing	Passed
Validation Testing	Passed
Performance Testing	Passed
User Acceptance Testing	Passed

Issues Identified and Resolved

Issue			Resolution Details			Status
Model path resolution errors on server run			Switched routes to absolute mappings	relative file path		Resolved
Large representation in formula card		float in GNI	Formatted coefficient notation (1.9e-6)		float to scientific	Resolved
Script activation blocked on Windows execution policies			Documented RemoteSigned execution commands in scripts		set-	Resolved

7. Project Demonstration

Local Execution Steps

To run the application locally on your system, follow these steps:

Open PowerShell and navigate to the code directory: `cd '5.Project Development Phase/code'`

Activate the virtual environment: `.\venv\Scripts\Activate`

Start the Flask server: `python app.py`

Open your browser and navigate to: `http://127.0.0.1:5000`

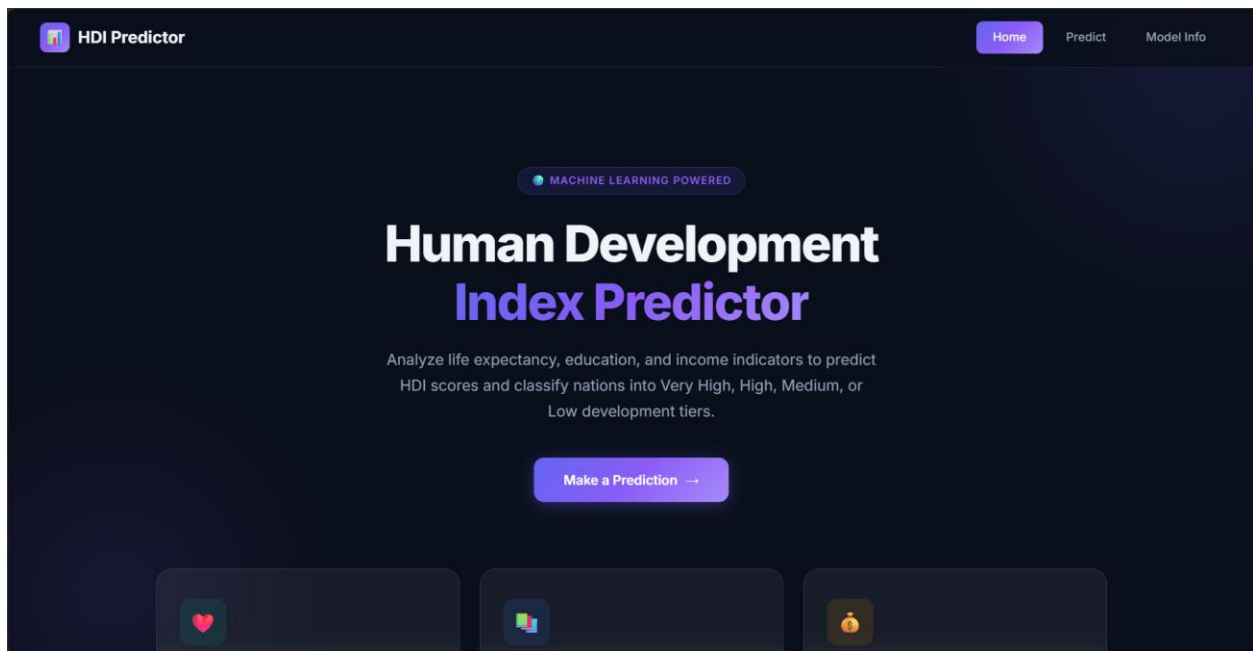


Figure 3: Home Page landing interface dashboard

The image shows the prediction form of the application. It is a dark gray rounded rectangle. At the top, there is a label 'Country' above a dropdown menu showing 'Country_A' with a downward arrow. Below this, there are four input fields arranged in a 2x2 grid. The first row has 'Life Expectancy (50-85 yrs)' with the value '72.5' and 'GNI per Capita (PPP \$500-75000)' with the value '42000'. The second row has 'Expected Yrs of Schooling (5-20)' with the value '14.5' and 'Mean Yrs of Schooling (3-15)' with the value '10.2'. At the bottom of the form is a large blue button with a white icon of a person and the text 'Predict HDI Score'.

Figure 4: prediction form

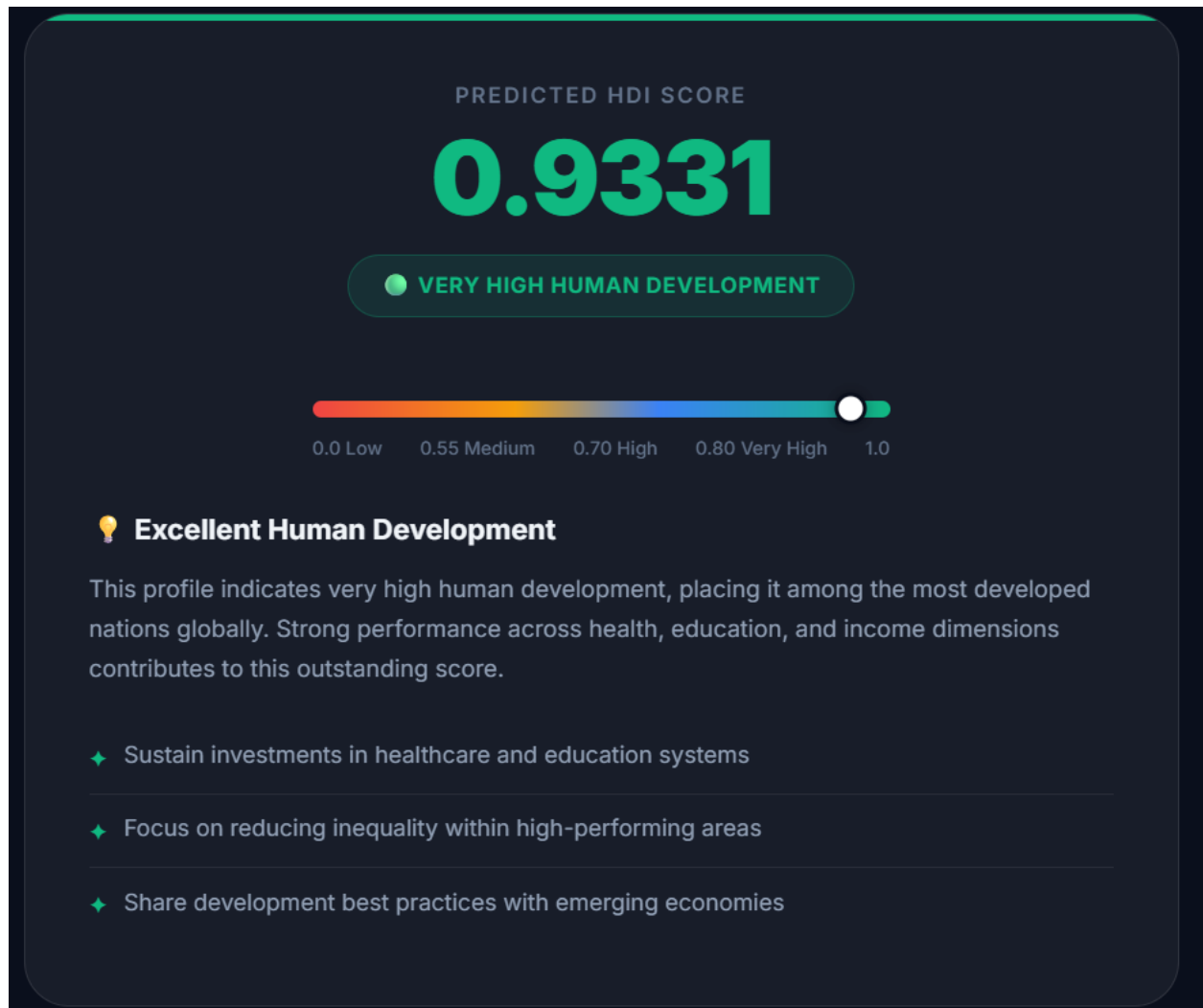


Figure 5: Prediction results and policy recommendation panels

Demonstrated Features

Real-time prediction calculation based on 4 developmental indicators.

Dynamic UNDP tier classification badges (Very High, High, Medium, Low).

Progress gauges mapped to predicted scores.

Insights engine displaying recommendations mapped to predicted tiers.

Model Info dashboard displaying metrics (R^2 , RMSE), feature weights, and EDA plots.

8. Achievements, Advantages, Limitations, Future Enhancements

Key Achievements

- Built an interactive prediction calculator for HDI scores.
- Trained a Linear Regression model achieving an R^2 score of 0.8834.
- Integrated predictions with a responsive Flask web server.
- Designed a dark-themed user interface utilizing glassmorphism cards.

Advantages of the System

- Enables real-time prediction calculation, bypassing annual reporting delays.
- Allows analysts to run interactive 'what-if' policy simulations.
- Displays visual gauges and badges to make outputs easy to interpret.
- Provides targeted recommendations based on predicted tiers.

Limitations

- Model is trained on historical datasets and does not reflect real-time global shifts.
- GNI coefficient is fitted on raw variables rather than log-scaled values.
- Application runs in a local environment and lacks user login databases.

Future Enhancements

- Train models on official UN datasets spanning 190+ countries over 30+ years.
- Integrate time-series forecasting to predict future HDI trends.
- Build database models to save user prediction history.
- Deploy the application to cloud environments like AWS.

9. Conclusion

Measuring human development is essential for analyzing global standards of living, health access, and knowledge acquisition. The United Nations Development Programme (UNDP) publishes the Human Development Index (HDI) annually. However, because these reports rely on complex data-compiling schedules, they are published retrospectively, resulting in significant delays. This delay prevents decision-makers from analyzing the immediate impacts of policy changes. The 'A Comprehensive Measure of Well-Being: Human Development Index (HDI)

Predictor' system addresses this challenge by providing a real-time prediction and policy recommendation dashboard using Python, Flask, and Scikit-learn.

Evaluation results demonstrate that the regression pipeline achieves a high R-squared accuracy score of 0.8834 and a Root Mean Squared Error (RMSE) of 0.0361. The trained model is serialized using the Pickle module and loaded into a responsive Flask web application. The frontend uses a premium, dark-mode glassmorphism layout featuring score progress gauges, status indicators, and targeted policy recommendations to help users evaluate long-term development plans.

With future enhancements such as cloud deployment, time-series forecasting, and database model integrations, this platform can evolve into a robust decision-support platform for global development.

10. Appendix

Appendix A: Software Requirements

Software	Version Required	Purpose
Operating System	Windows 10/11 or macOS/Linux	Host execution environment
Python	Version 3.10+	Programming language execution
Flask	Version 3.1.3	Backend routing framework
Scikit-learn	Version 1.9.0	Model prediction library
Pandas & NumPy	Latest standard versions	Data manipulation
Seaborn & Matplotlib	Latest standard versions	Visual graphs plotting

Appendix B: Hardware Requirements

Hardware Parameter	Minimum Specification	Recommended Specification
Processor	Dual Core 2.0 GHz	Quad Core 3.0 GHz+ (Intel i5/i7)
Memory (RAM)	4 GB	8 GB or higher
Storage Space	500 MB free space	10 GB free space (for dataset files)
Network	Not required for local execution	Required for package setup

Appendix C: Key Algorithms & Technologies

Linear Regression: Simple, highly interpretable baseline regression fitting target HDI values using standard features. Label Encoding: Preprocesses text categories into integer IDs. Pickle Module: Serializes the regression model and encoder variables into reusable artifacts.

Appendix D: Project Folder Structure

HumanDevelopmentIndex/

[illegible]

Appendix E: Performance Metrics Comparison

Model Evaluation Model	R ² Accuracy Score	RMSE Error
Linear Regression (Baseline)	0.8834	0.0361
Decision Tree Regressor	0.8142	0.0452
K-Neighbors Regressor	0.7811	0.0491

Appendix F: References

- Python official reference documentation: <https://docs.python.org/3/>
- Flask web framework documentation: <https://flask.palletsprojects.com/>
- Scikit-learn documentation: <https://scikit-learn.org/>
- Seaborn statistical plots: <https://seaborn.pydata.org/>
- Matplotlib graphics layout: <https://matplotlib.org/>
- UNDP Human Development reports index: <https://hdr.undp.org/>

GitHub Repository: <https://github.com/Jaswanth258/HDI>